

4

Decidir y depurar

Cómo hacer que un workflow decida por qué ruta va cada pedido de Mercado Austral y cómo encontrar, explicar y corregir sus errores con las herramientas de n8n.

APRENDIZAJE ESPERADO 4 · TEXTUAL DEL PLAN FORMATIVO

IMPLEMENTAR LÓGICA CONDICIONAL Y ESTRUCTURAS DE CONTROL EN WORKFLOWS, APLICANDO BUENAS PRÁCTICAS DE DEBUGGING Y TRAZABILIDAD.

PLAN FORMATIVO

PF1821 · Construcción de Agentes y Automatización con Herramientas Low Code

MÓDULO

MA04560 · FUNDAMENTOS DE N8N Y MANIPULACIÓN DE DATOS · 18 h

CASO DE ESTUDIO

Mercado Austral, empresa ficticia

TIEMPO DE LECTURA

15 minutos, más la autocomprobación

Qué vas a lograr

Al terminar esta lectura tendrás la base para demostrar los criterios de evaluación del aprendizaje esperado 4, textuales del plan formativo:

- 4.1 CONFIGURA WORKFLOWS CON MÚLTIPLES RUTAS CONDICIONALES.
- 4.2 UTILIZA OPERADORES LÓGICOS Y COMPARACIONES EN EXPRESIONES N8N.
- 4.3 APLICA HERRAMIENTAS DE DEBUGGING PARA IDENTIFICAR ERRORES BÁSICOS EN WORKFLOWS.

Cómo está organizada

- | | |
|------------------------------|--|
| 1 If o Switch | 6 Contar ítems |
| 2 Operadores | 7 Trazabilidad |
| 3 Enrutar datos | 8 Buenas prácticas de prueba |
| 4 Casos borde | + En síntesis, práctica, autocomprobación y glosario |
| 5 Herramientas de depuración | |

Contenidos del plan y dónde se tratan

Contenido del plan formativo (textual)	Dónde
4. LÓGICA CONDICIONAL Y DEBUGGING EN N8N	Antes de empezar
OPERADORES Y EXPRESIONES COMPLEJAS.	Sección 2
ROUTING DE DATOS.	Secciones 1 y 3
MANEJO DE CASOS EDGE.	Sección 4
HERRAMIENTAS DE DEBUG EN N8N.	Secciones 5 y 6
LOGS Y TRAZABILIDAD.	Sección 7
MEJORES PRÁCTICAS DE TESTING.	Sección 8

PARA COMENZAR

Antes de empezar

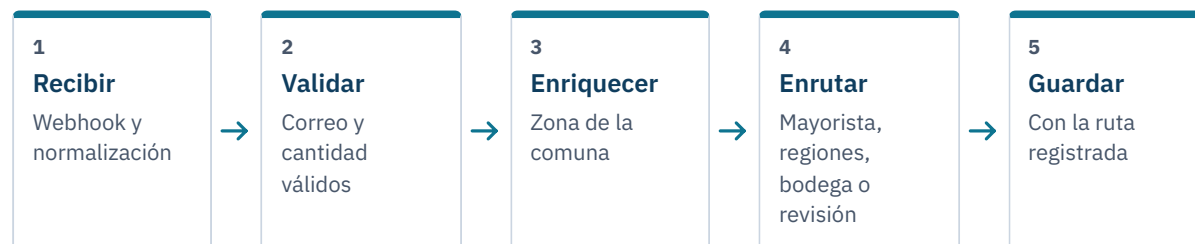
CONTENIDO DEL PLAN 4. LÓGICA CONDICIONAL Y DEBUGGING EN N8N

Hasta ahora tus workflows hacen siempre lo mismo con cada pedido. Los procesos reales no son así: un pedido mayorista va a ventas, uno de regiones va a despacho regional y uno con datos inválidos no debería guardarse. Este aprendizaje trata de dos habilidades que

van juntas. La primera es **decidir**: que el workflow elija la ruta de cada ítem según sus datos. La segunda es **depurar**: encontrar por qué una decisión salió mal, explicarlo y corregirlo.

En Mercado Austral, un practicante dejó un workflow que enrutaba los pedidos. Funcionó una semana y después empezó a fallar: pedidos perdidos, mayoristas tratados como minoristas y ejecuciones en rojo. Ventas está molesta. Con lo que aprendas aquí vas a poder rescatarlo en la actividad 2.

EL WORKFLOW DE ENRUTAMIENTO



1 If o Switch

CONTENIDO DEL PLAN ROUTING DE DATOS.

El **enrutamiento de datos** (*routing*) es enviar cada ítem por un camino distinto según sus valores. n8n tiene dos nodos principales para hacerlo. **If** evalúa una condición y tiene dos salidas: **verdadero** y **falso**. **Switch** tiene varias salidas, una por regla, y envía cada ítem a la salida cuya regla se cumple.

Usa If cuando la pregunta tiene respuesta de sí o no: ¿el pedido es válido?, ¿el monto supera el límite? Usa Switch cuando hay tres o más destinos posibles. Switch trabaja en dos modos: en **Rules** defines una regla por salida con los mismos operadores que If; en **Expression** una sola expresión devuelve el número de la salida. El modo Rules es más fácil de leer y de revisar, y es el recomendado para empezar.

EJEMPLO · MERCADO AUSTRAL · Dos decisiones en el mismo workflow

En el workflow de Mercado Austral, un **If** valida cada pedido: si el correo contiene "@" y la cantidad es mayor que 0, sigue por la salida verdadera; si no, va por la falsa a la tabla `pedidos_rechazados`. Después, un **Switch** con tres reglas decide entre ventas, despacho regional y bodega de Santiago.

2 Operadores

CONTENIDO DEL PLAN OPERADORES Y EXPRESIONES COMPLEJAS.

Cada condición compara un valor con otro usando un **operador**. En If y Switch primero eliges el tipo de dato (texto, número, fecha, booleano, array u objeto) y después el operador: es igual a, no es igual a, contiene, comienza con, es mayor que, es mayor o igual

que, está vacío, existe. Varias condiciones se combinan con **AND**, que exige que se cumplan todas, o con **OR**, que exige al menos una.

Dentro de una expresión se usan los operadores de JavaScript. Permiten escribir condiciones complejas en un solo campo, o calcular un valor que después se compara.

Operador	Significado	Ejemplo
<code>===</code>	Igual en valor y en tipo	<code>\$json.tipo_cliente === 'mayorista'</code>
<code>!==</code>	Distinto	<code>\$json.zona !== 'RM'</code>
<code>>=</code> <code><</code>	Mayor o igual, menor	<code>\$json.total >= 150000</code>
<code>&&</code>	Y: se cumplen las dos	<code>\$json.cantidad > 0 && \$json.email.includes('@')</code>
<code> </code>	O: basta con una	<code>\$json.tipo_cliente === 'mayorista' \$json.total >= 150000</code>
<code>!</code>	Negación	<code>!\$json.email</code> es verdadero si el correo está vacío
<code>? :</code>	Si, entonces, si no	<code>\$json.total >= 150000 ? 'alto' : 'normal'</code>

EJEMPLO · MERCADO AUSTRAL · La regla del mayorista

Un pedido es mayorista si el cliente es mayorista o si el total es de \$150.000 o más. En la regla del Switch son dos condiciones combinadas con **OR**: `tipo_cliente` (texto) es igual a "mayorista" y `total` (número) es mayor o igual que 150000. Como expresión: `{{ $json.tipo_cliente === 'mayorista' || $json.total >= 150000 }}`.

ERROR FRECUENTE · El número que era texto

Si `total` llega como "219000", entre comillas, la comparación numérica no se cumple y un pedido mayorista cae en la ruta equivocada. En JavaScript, `"219000" === 219000` es falso porque los tipos son distintos. Declara `total` como Number en Edit Fields; como alternativa, la opción *Convert types where required* del nodo convierte los tipos al comparar.

3 Enrutar datos

CONTENIDO DEL PLAN ROUTING DE DATOS.

Enrutar bien es más que escribir condiciones: es decidir **el orden** en que se evalúan. En el modo Rules, el Switch envía cada ítem a la **primera** regla que se cumple, salvo que actives la opción para enviarlo a todas las que se cumplan. Por eso las reglas se ordenan de la más específica o prioritaria a la más general.

Salida	Regla	Destino
0 · mayorista	<code>tipo_cliente</code> es igual a "mayorista" OR <code>total</code> es mayor o igual que 150000	Ventas

Salida	Regla	Destino
1 · regiones	zona es igual a "Regiones"	Despacho regional
2 · bodega RM	tipo_cliente es igual a "minorista"	Bodega de Santiago
respaldo	Lo que no calza con ninguna regla	Revisión manual

EJEMPLO · MERCADO AUSTRAL · Un mayorista de Temuco

El pedido del Almacén El Sol, de Temuco, es mayorista y de regiones: cumple las reglas 0 y 1. Como la regla 0 va primero, el pedido se envía a **ventas**, que es lo que Mercado Austral quiere: ventas coordina después el despacho. Si el orden fuera otro, iría a despacho regional sin pasar por ventas.

IDEA CLAVE

Cuando dos reglas pueden cumplirse a la vez, el orden es parte de la lógica del negocio. Escríbelo en una nota del canvas para que quien mantenga el workflow no lo cambie sin querer.

4 Casos borde

CONTENIDO DEL PLAN MANEJO DE CASOS EDGE.

Un **caso borde** (*edge case*) es una entrada válida pero poco común, o una que nadie previó, que hace fallar una lógica que funciona con los casos normales. En los pedidos de Mercado Austral aparecen cinco tipos: **campos vacíos** (un pedido sin correo), **tipos equivocados** (una cantidad escrita "3 " con un espacio), **valores inesperados** (un `tipo_cliente` "distribuidor" que no es ni minorista ni mayorista), **duplicados** (el mismo pedido enviado dos veces) y **valores extremos** (una cantidad en cero o un monto fuera de lo habitual).

Se manejan con tres estrategias. **Validar temprano**: un If al comienzo separa lo que no sirve antes de procesarlo. **Normalizar**: `trim()`, `toLowerCase()` y la conversión de tipos eliminan diferencias que no importan. Y **no perder nada en silencio**: en el Switch, la **salida de respaldo** (*Fallback Output*, opción *Extra Output*) recibe todo lo que no calza con ninguna regla y lo envía a revisión manual.

EJEMPLO · MERCADO AUSTRAL · El pedido de Isla de Pascua

Un pedido con `tipo_cliente` "distribuidor" llegó y desapareció: no quedó en ninguna tabla y la ejecución salió en verde. Ninguna regla aceptaba ese valor y el Switch no tenía salida de respaldo, así que lo descartó sin avisar. Con *Fallback Output* activado y conectado a una rama de revisión manual, el pedido queda registrado y alguien decide qué hacer.

ERROR FRECUENTE · Arreglar el caso y no la regla

Agregar "distribuidor" a la regla de minoristas resuelve este pedido, pero el próximo valor inesperado se vuelve a perder. La corrección general es la salida de respaldo: que lo desconocido siempre tenga un destino.

5 Herramientas de depuración

CONTENIDO DEL PLAN HERRAMIENTAS DE DEBUG EN N8N.

Depurar (*debugging*) es encontrar la causa de un error y corregirla. n8n ofrece cuatro herramientas que conviene usar en este orden:

- **Historial de ejecuciones.** Cada ejecución queda registrada con su estado, su hora y los datos de cada nodo. Al abrirla ves el workflow tal como corrió: qué nodos se ejecutaron, cuáles fallaron y qué recibió y entregó cada uno. En los ajustes del workflow se decide qué ejecuciones se guardan.
- **Ejecutar un solo nodo.** El botón para ejecutar solo ese paso prueba un nodo con los datos que tiene a la entrada, sin correr todo el workflow.
- **Datos fijados.** Permiten reproducir el caso que falló una y otra vez mientras pruebas la corrección.
- **Mensajes de error completos.** Un nodo en rojo muestra el mensaje y sus detalles; casi siempre dicen qué campo o qué valor causó el problema.

EJEMPLO · MERCADO AUSTRAL · Encontrar la causa de "nadie tiene correo"

Todos los pedidos salen por la rama de datos inválidos. En el historial abres una ejecución y miras el If: la vista previa de su condición muestra `undefined`. La expresión dice `{{ $json.Email }}`, con mayúscula, y el campo se llama `email`. Se corrige la expresión, se fija un pedido de prueba, se ejecuta el nodo solo y ahora sale por la rama verdadera.

6 Contar ítems

CONTENIDO DEL PLAN HERRAMIENTAS DE DEBUG EN N8N.

La prueba más útil para depurar es también la más simple: **¿cuántos ítems entran y cuántos salen de cada nodo?** Después de una ejecución, el canvas muestra la cantidad de ítems sobre cada conexión, y cada panel indica cuántos hay. Si los números no cuadran con lo que esperas, el problema está en ese nodo.

Cada tipo de nodo tiene una cuenta esperada. Un Edit Fields entrega los mismos ítems que recibe. Un Filter o un If pueden entregar menos por una salida, pero la suma de sus salidas debe ser igual a la entrada. Un Switch con salida de respaldo nunca debería perder ítems. Un Merge por campo en común entrega, como máximo, tantos ítems como la entrada principal.

EJEMPLO · MERCADO AUSTRAL · Dos cuentas que delatan dos fallas

Con 6 pedidos de prueba fijados: el Switch recibe 6 ítems y la suma de sus salidas es 5. Falta uno, el "distribuidor", y la causa es la falta de salida de respaldo. En el Merge que agrega la zona, entran 6 pedidos y salen cientos de ítems: está en modo de todas las combinaciones y cada pedido se repite una vez por comuna.

ERROR FRECUENTE · Confiar en el verde

Una ejecución en verde solo significa que ningún nodo lanzó un error. No significa que no se perdieron datos ni que las rutas son correctas. Cuenta los ítems en cada prueba, sobre todo después de un If, un Switch, un Filter o un Merge.

7 Trazabilidad

CONTENIDO DEL PLAN LOGS Y TRAZABILIDAD.

La **trazabilidad** es poder reconstruir qué pasó con cada dato: por dónde pasó, qué decisión se tomó y en qué ejecución. Los **logs** son los registros que lo permiten. En n8n hay dos fuentes: el historial de ejecuciones, que registra la corrida completa, y los campos que tú agregas a cada registro guardado.

Tres prácticas hacen que un workflow sea trazable. **Guardar la decisión:** en cada rama, un Edit Fields agrega un campo `ruta` con el nombre de la salida. **Guardar la ejecución:** el campo `id_ejecucion` con `{{ $execution.id }}` conecta cada fila con su ejecución en el historial. Y **manejar los errores:** en los ajustes de un nodo, la opción *On Error* con *Continue (using error output)* agrega una salida de error para registrar el ítem que falló sin detener todo el workflow. Además, un **workflow de errores**, que empieza con el nodo *Error Trigger* y se asigna en los ajustes del workflow principal, recibe un aviso cada vez que una ejecución falla.

EJEMPLO · MERCADO AUSTRAL · La ejecución que ya no se detiene

Supabase rechaza el pedido con `cantidad "3"` y el workflow completo se detiene: los otros cinco pedidos tampoco se guardan. Con *On Error* en *Continue (using error output)*, el pedido con error sale por una salida aparte hacia `pedidos_rechazados`, con el mensaje del error, y los demás se guardan. Cada fila lleva su `ruta` y su `id_ejecucion`.

8 Buenas prácticas de prueba

CONTENIDO DEL PLAN MEJORES PRÁCTICAS DE TESTING.

Probar un workflow no es ejecutarlo una vez y ver que sale en verde. Estas prácticas marcan la diferencia:

1. **Datos de prueba que incluyan casos sucios.** Un conjunto fijo con un caso normal por ruta y los casos borde conocidos.

2. **Probar cada ruta.** Todas las salidas del If y del Switch, incluida la de respaldo, deben recibir al menos un caso de prueba.
3. **Nombrar los nodos por lo que hacen.** "Validar pedido" y "Enrutar por tipo" se entienden en el historial; "If1" y "Switch2", no.
4. **Cambiar una cosa a la vez** y volver a correr todos los casos después de cada corrección, para confirmar que no se rompió otra parte.
5. **Anotar cada falla en una bitácora:** síntoma, cómo se detectó, causa, corrección, prueba y aprendizaje.

EJEMPLO · MERCADO AUSTRAL · Los seis pedidos de prueba

Para el workflow de enrutamiento, Mercado Austral usa seis pedidos fijados: uno minorista de Santiago, uno mayorista por tipo, uno **mayorista por monto** (\$219.000), uno **de Temuco**, uno con `tipo_cliente` "**distribuidor**" y uno con `cantidad` escrita "**3**". Con esos seis se prueban todas las rutas y los cuatro casos borde que ya causaron problemas.

En síntesis

- If decide entre dos caminos; Switch, entre varios, en modo Rules o Expression.
- Las condiciones usan operadores de comparación y se combinan con AND (todas) u OR (al menos una); en expresiones, con `&&`, `||` y `!`.
- En el Switch gana la primera regla que se cumple: el orden de las reglas es parte de la lógica.
- Los casos borde se manejan validando temprano, normalizando y con una salida de respaldo que no pierde nada.
- Para depurar: historial de ejecuciones, ejecutar un nodo, datos fijados y mensajes de error completos.
- Contar ítems de entrada y de salida en cada nodo revela la mayoría de las fallas; el verde no garantiza nada.
- La trazabilidad se logra guardando la ruta y el identificador de la ejecución, con salidas de error y un workflow de errores.
- Se prueba con casos sucios, cada ruta, nodos bien nombrados, un cambio a la vez y una bitácora.

SIGUIENTE PASO

Para practicar

- **Actividad 2, "Rescate del workflow roto".** Cinco misiones, cada una con una falla escondida: detectarla, explicar la causa y corregirla, con tu bitácora de depuración.
- **Video interactivo "Expresiones y depuración".** Condiciones con AND y OR, salida de respaldo del Switch y conteo de ítems a la salida de un Merge.
- **Documentación oficial.** La de n8n (docs.n8n.io) describe los nodos If y Switch, el manejo de errores y los workflows de errores.

Autocomprobación

Responde con tus palabras antes de mirar las respuestas. Si dudas en alguna, vuelve a la sección que se indica.

- 1 Con la condición "`tipo_cliente` es igual a mayorista **AND** `total` es mayor o igual que 150000", ¿por qué sale un pedido minorista de \$219.000? ¿Cómo corriges la regla si basta con que se cumpla una de las dos?
- 2 Entran 6 pedidos al Switch y sus salidas suman 5, y la ejecución terminó en verde. ¿Qué pasó y cuál es la corrección general?
- 3 ¿Qué dos campos agregas a cada pedido guardado para que sea trazable, y con qué expresión obtienes uno de ellos?

RESPUESTAS

1. Sale por la rama falsa, porque con AND deben cumplirse las dos condiciones y el cliente no es mayorista. Si basta con una, las condiciones se combinan con OR (sección 2).
2. Un pedido no calzó con ninguna regla y el Switch, sin salida de respaldo, lo descartó sin error. La corrección general es activar *Fallback Output* con *Extra Output* y conectarla a una rama de revisión manual (secciones 4 y 6).
3. `ruta`, con el nombre de la salida por la que pasó, e `id_ejecucion`, que se obtiene con `{{ $execution.id }}` y permite encontrar la ejecución en el historial (sección 7).

Glosario

Caso borde	Entrada poco común o no prevista, como un campo vacío o un valor inesperado, que puede romper la lógica.
Error Trigger	Nodo que inicia un workflow de errores cada vez que falla una ejecución de otro workflow.
Historial de ejecuciones	Registro de cada vez que corrió un workflow, con su estado y los datos de cada nodo.
If	Nodo que evalúa una condición y envía cada ítem por la salida verdadera o por la falsa.
Operador lógico	Operador que combina condiciones: AND exige todas, OR al menos una y la negación invierte el resultado.
Salida de respaldo	Salida extra del Switch (<i>Fallback Output</i>) que recibe los ítems que no cumplen ninguna regla.
Switch	Nodo que envía cada ítem a una de varias salidas según reglas o según el resultado de una expresión.
Trazabilidad	Capacidad de reconstruir qué pasó con cada dato: su ruta, su decisión y su ejecución.